Task Offloading and Resource Allocation in Vehicular Networks: A Lyapunov-Based Deep Reinforcement Learning Approach

Anitha Saravana Kumar, *Student Member, IEEE*, Lian Zhao, *Senior Member, IEEE*, and Xavier Fernando, *Senior Member, IEEE*

***Abstract*—Vehicular Edge Computing (VEC) has gained popularity due to its ability to enhance vehicular networks. VEC servers located at Roadside Units (RSUs) allow low-power vehicles to offload computation-intensive and delay-sensitive applications, making it a promising solution. However, optimal resource allocation between edge servers is a complex issue due to vehicle mobility and dynamic data traffic. To address this issue, we propose a Lyapunov-based Multi-Agent Deep Deterministic Policy Gradient (L-MADDPG) method that jointly optimizes computing task distribution and radio resource allocation to minimize energy consumption and delay requirements. We evaluate the trade-offs between the performance of the optimization algorithm, queuing model, and energy consumption. We first examine delay, queue and energy models for task execution at the vehicle or RSU, followed by the L-MADDPG algorithm for jointly optimizing task offloading and resource allocation problems to reduce energy consumption without compromising performance. Our simulation results show that our algorithm can reduce energy consumption while maintaining system performance compared to existing algorithms.**

***Index Terms*—Lyapunov optimization, multi-agent DDPG, reinforcement learning, resource management, VEC, vehicle edge computing, vehicular networks.**

I. Introduction

NOWADAYS, with the popularity of the Internet of Vehicles (IoV) and intelligent transportation system (ITS) concepts, connected vehicles need to become more and more intelligent with increased computing power. Modern vehicles are expected to execute innovative in-vehicle applications such as image-assisted navigation, multimedia entertainment, and even autonomous driving [1], [2]. These smart in-vehicle applications, especially those related to autonomous navigation, will consume many computational resources and require low execution latency. Indeed it is challenging to achieve all these functionalities purely in the vehicles due to their limited computing capability. Therefore, to assist these intelligent vehicular applications, vehicle edge computing (VEC) servers are increasingly deployed densely on the macro-assisted eNodeB (VeNB), roadside units (RSU) [3].

The VEC approach is more suitable for dealing with delay-sensitive tasks with low latency than cloud computing. It can effectively distribute the enhanced services among the RSU or the vehicle, significantly improving the computing time and operating efficiency. Therefore, VEC-based optimization schemes that can optimize task offloading and resource allocation more comprehensively from the system perspective are under intense research [4].

Researchers have focused on improving the VEC's architecture, and functional modules for task offloading and resource management among connected vehicles [5]. It is well known that future interconnected vehicular networks require convergence of communication, computing, and control [6]. The communication networks may achieve better performance and higher efficiency with the help of artificial intelligence algorithms [7], [8]. Since there is no labelled data available, integrating deep reinforcement learning (DRL) is a practical step toward intelligent, autonomous, and reliable communication networks in a vehicle-to-everything (V2X) environment [9].

Lyapunov functions are used extensively in control theory to ensure system stability, assuming the system state is a multi-dimensional vector. A Lyapunov function is a non-negative scalar measure of this multi-dimensional state [10], [11]. Here, system stability is achieved by taking control actions that move the Lyapunov function towards zero [12]. We apply the Lyapunov optimization to obtain queue stability during resource allocation in edge-assisted vehicular networks.

A. Related Works

- *VEC servers in resource allocation:* Intensive computing is expected to incur in future vehicular networks due to IoV and ITS concepts, and often, the computing capabilities of stand-alone vehicles are inadequate [44], [45]. As the technology for autonomous vehicles matures, prompt information exchanges with massively connected and autonomous vehicles are required. Hence, many researchers have focused on VEC-based resource allocation and optimization schemes. The objective is mainly on offloading computing tasks to a VEC server to achieve low latency

Manuscript received 20 December 2022; revised 21 March 2023; accepted 10 April 2023. Date of publication 3 May 2023; date of current version 17 October 2023. This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Grant RGPIN-2020-04678. The review of this article was coordinated by Dr. Tao Dusit Niyato. *(Corresponding author: Lian Zhao.)*

The authors are with the Department of Electrical, Computer, Biomedical Engineering, Toronto Metropolitan University, Toronto, ON M5B 2K3, Canada (e-mail: anitha.saravanakumar@torontomu.ca; l5zhao@torontomu.ca; fernando@torontomu.ca).

Digital Object Identifier 10.1109/TVT.2023.3271613

0018-9545 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

and energy efficiency goals. A few researchers have studied the architecture and functional modules of the VEC servers. Some have discussed the potential challenges and prospects of optimizing VEC-based task offloading. For instance, in [13], the resource allocation problem is modelled as a mixed-integer nonlinear programming problem and solved by applying a joint RSU selection and resource allocation using the VEC system. In [14], an adaptive learning algorithm based on the multi-armed bandit theory is employed to minimize the task offloading delay in VEC systems. In [15], the authors proposed a collaborative task offloading and output transmission mechanism to guarantee low latency and load-balancing. The authors in [16] addressed a delay-based task offloading algorithm to minimize the total network delay based on road conditions. The authors in [17] introduced novel wireless distributed architectures that embed the edge computing capability inside software-defined vehicular networking infrastructure, enabling dynamic eco-routing with ultra-low latency and mobile data-driven orchestrations.

- *Lyapunov optimization in resource allocation:* Many researchers have investigated the dynamic resource allocation strategies in standard wireless networks. However, vehicular networks have distinct characteristics. They demand computationally efficient algorithms and a stable solution in a fast-varying dynamic environment. The Lyapunov optimization algorithm has been attempted to address this need in vehicular networks. For example, the authors of [20] proposed an online vehicular caching design by optimizing network energy efficiency using the Lyapunov framework. The authors achieved a dynamic cross-layer transmission to meet the QoS requirements of different users. In [21], the authors used a combination of game theory and Lyapunov optimization for cloud-based content delivery networks in the vehicular environment. Also, the authors have deployed Lyapunov optimization for resource allocation for heterogeneous users. The authors proposed a dynamic cross-layer transmission strategy to increase vehicle and static users' total throughput while satisfying the different quality of service requirements [22]. Hung et al. [23] suggested a task offloading approach for a mobile edge computing-assisted vehicle platoon using the Lyapunov optimization algorithm. Also, the authors described a vehicle-centric framework using Lyapunov optimization techniques to achieve low-latency communication among vehicle-to-vehicle networks.

  The Lyapunov optimization approach uses a quadratic function to stabilize a system [38]. More than stability, Lyapunov's methods can be coupled with convex optimization techniques to produce robust and concrete results [18]. The other advantage of Lyapunov optimization is that it transforms a stochastic optimization problem into a series of deterministic optimization problems, which we plan to take advantage of. However, due to the stochastic nature of data arrival in a vehicular environment, there is a trade-off between throughput and delay considering the average long-term perspective [19]. Although other well-known heuristics and genetic optimization techniques exist besides Lyapunov, these algorithms are more iterative and less stable in a dynamic environment [39]. Since Lyapunov can be used for constraint optimization, the mean value can be determined in a stochastic system without having to predict future random events. We handle this issue with the MADDPG policy.
- *DRL techniques in resource allocation:* The dynamic nature of connected vehicles pushes network management to a new level where the DRL can play a vital role. Researchers have developed adaptive communication and control policies using DRL methods. Recently, DRL techniques are increasingly trending to solve cross-layer optimization involving access control and resource allocation [24]. In [25], authors consider an efficient task offloading scheme based on trajectory prediction for VEC using deep Q-networks to address the dynamic nature of vehicular networks. Also, the work focuses on vehicles that cooperate with RSUs or the surrounding vehicles for task processing. However, the impact of the vehicle's mobility on the channel state change is essential during the task-offloading process. In [26], authors use a shared task type and vehicle speed-aware task offloading for resource allocation strategy using the MADDPG algorithm. In [27], authors optimize task offloading using DRL techniques to reduce task latency and energy costs. Also, in [28], DRL provides a unique decision algorithm to perform intelligent vehicular-controlled services based on an edge computing model.

  However, most existing works have studied only vehicular networks supported by VEC-mounted at base stations. Efficient resource allocation that supports minimum energy consumption under queue stability in vehicular networks with VEC-mounted base stations still needs effort. That is the focus of this work. Our proposed Lyapunov-based MADDPG algorithm addresses the problem of instability in the multi-agent environment by utilizing centralized training and distributed execution.

B. Contributions

This paper investigates a Lyapunov-based multi-agent resource allocation scheme for VEC-assisted vehicular networks. We jointly optimize the strategies for task scheduling and resource allocation. We minimize the system's energy consumption by considering the local vehicle and edge server with a performance guarantee. Since our problem is a non-convex issue, we adopt reinforcement learning (RL) methods to achieve distributed and cooperative real-time resource allocation under the constraints of reducing the system energy consumption and guaranteeing queue stability. The main contributions are listed as follows:

- Our work addresses the problem of energy-efficient task offloading and resource allocation in VEC-assisted vehicular networks with multiple RSUs and a group of vehicles. To this end, we propose a systematic framework that allows

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

vehicles to keep a local computation queue and a local offloading queue for each task. Tasks generated by the vehicle can be processed locally or offloaded to the edge server. The framework considers multiple energy optimization decision variables, such as where to place the vehicle-generated tasks, the transmission power constraint in task offloading, the local CPU cycle frequency for processing computation tasks, the number of computation tasks to be offloaded, and the allocation computation resource of edge servers.

- We model the transformed problem as a multi-agent problem, with each vehicle acting as an agent. Due to the variable vehicle association pattern, the formulated problems are coupled and non-convex. To overcome this issue, we divide the deterministic optimization problem into two distinct problems. We implement the proposed Lyapunov-based multi-agent deep deterministic policy gradient (L-MADDPG) algorithm on each vehicle (agent). Since each VEC server associated with RSU calculates the computation ratio based on the demand from its connected vehicles, the problem of computation resource allocation from RSU is decoupled, and it is a crucial optimization variable for determining resource allocation policies.
- Our L-MADDPG algorithm can devise efficient task scheduling and resource allocation strategies dynamically, without relying on prior system state information. We train the L-MADDPG model offline to ensure that each agent schedules and allocates resources efficiently. Our experimental results show that the proposed L-MADDPG algorithm dynamically adjusts decision variables to minimize system energy consumption while maintaining queue stability and meeting latency requirements. We believe that our proposed framework and L-MADDPG algorithm can significantly reduce energy consumption and improve the performance of VEC-assisted vehicular networks.

C. Organization of the Paper

The rest of the paper is structured as follows. We elaborate on the system model and problem formulation in Section II. In Section III, we discuss the Lyapunov optimization. Section IV details the edge server computation and proposed Lyapunov-based multi-agent deep deterministic policy gradient solution. Section V implements the proposed algorithm and validates its effectiveness and efficiency by numerical results. We draw our conclusion in Section VI.

II. System Model and Problem Formulation

A. VEC-Assisted Vehicular Network Model

In this section, the VEC-assisted vehicular network architecture is introduced. In the VEC environment, we implement a discrete slot model expressed as $\mathcal{T} = \{1, \ldots, t \cdots, T)\}$ and the duration of each time slot is defined as δ. A highway condition is considered, as shown in Fig. 1. We consider a cognitive-radio-enabled vehicular network with V RSUs along

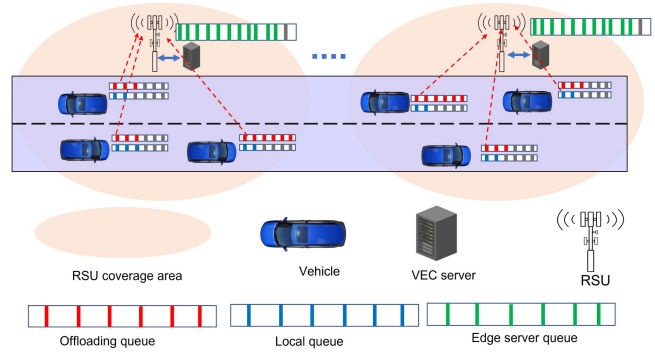


Fig. 1. Considered VEC-assisted vehicular network scenario.

the road. Each RSU[1] is equipped with a VEC server. Hence, there are V VEC servers denoted by $\mathcal{V} = \{1, \ldots, v \cdots, V\}$. Each VEC server v is associated with the group of vehicles expressed by $\mathcal{K}^{v2} = \{1, \ldots, k, \cdots K^v\}$. As mentioned in [29] and [30], we assume that once the vehicle enters a VEC's service area, it will agree to participate in the VEC system. We also assume that the vehicles can simultaneously transmit and compute data and that an adequate power supply is available from onboard energy sources. Vehicles update their status information (i.e., their speed, position, task queue length, current computation status, and wireless resources) periodically to VEC servers. Each vehicle attempts to minimize energy costs through optimal offloading policies.

Table I summarizes the significant variables used in the system model.

B. Task and Communication Model

At the start of each time slot, every vehicle generates a computation task. $D_{v,k}(t)$ denotes the size of this generated computation task at vehicle k, which is related to server v. Its maximum delay tolerance is $\tau_{v,k}(t)$. Following [31], we adopt a binary offloading approach for the computation task. Let $\lambda_{v,k}(t) \in \{0, 1\}$ represent the offloading decisions, where $\lambda_{v,k}(t) = 0$ denotes the task is locally processed and $\lambda_{v,k}(t) = 1$ means that the task will be offloaded for computation.

We assume that the transmissions are done on orthogonal channels. Let $p_{v,k}(t)$ represent the wireless transmission power of vehicle k with a maximum value $P^l_{\max}(t)$ and σ^2 is the background noise power. We define $g_{v,k}(t) = G^{-l}_{v,k}$ as the channel gain between the VEC server v and the vehicle k, where $-l$ is the path loss exponent and $G^{-l}_{v,k}$ denote the distance between the VEC v and the vehicle k. The channel bandwidth is represented by B. Thus, the transmission rate is expressed by,

$$R_{v,k}(t) = B \log\left(1 + \frac{p_{v,k}(t) g_{v,k}(t)}{\sigma^2}\right). \tag{1}$$

[1]Contrast to some other authors, we don't call the computers in vehicles as VEC servers.

[2]In this paper, we added (t) at the end of some notations to distinguish the fixed parameters from the time-varying parameters. Yet the vehicular network is assumed to be static during each time slot [41].

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS

Symbol	Description
$\mathcal{V}$	The RSU Set
$\mathcal{K}^v$	Set of vehicles associated with the RSU v
$R_{v,k}(t)$	Transmission rate
$p_{v,k}(t)$	Transmission power
$g_{v,k}(t)$	Channel power gain
$f^l_{v,k}(t)$	Local CPU-cycle frequency
σ^2	Noise power
δ	Duration of the time slot
$\tau_{v,k}(t)$	The task's maximum tolerated delay
$f_v(t)$	Computational resources assigned by VEC v
$b_{v,k}$	Number of CPU-cycle needed to process a bit of computational task
ξ_k	Computation energy efficiency coefficient of vehicle k
ζ_v	Computation energy efficiency coefficient of VEC v
$\tau^l_{v,k}(t)$	The local processing delay in vehicle k
$\tau^o_{v,k}(t)$	The task upload delay in VEC v
$\tau^c_{v,k}(t)$	The task processing delay in VEC v
$\tau^{tot}_{v,k}(t)$	The total latency of computing offloading
B	Channel bandwidth of RSU
$\varphi_{v,k}(t)$	Computation time ratio that VEC v allocates to the vehicle k
$D_{v,k}(t)$	Size of the arrived task at vehicle k
$D^l_{v,k}(t)$	Size of the *locally* computed tasks
$D^o_{v,k}(t)$	Size of the *offloaded* tasks
$D^c_{v,k}(t)$	Size of the tasks processed by the edge server
$Q^l_{v,k}(t)$	Local computation queue length
$Q^o_{v,k}(t)$	Offloading queue length
$Q^c_{v,k}(t)$	Queue length in the edge server

C. Delay Model

In vehicular network scenarios, we consider the tradeoff between energy consumption and service latency to balance energy consumption and delay. To guarantee minimal service delay, we begin by carefully crafting delay models for local and offloading computations. Let $b_{v,k}$ represent the number of CPU cycles required to calculate a bit of computation tasks and $f^l_{v,k}(t)$ is the computing resources allocated by the vehicle. In the case of local computing, the computation task is processed utilizing the vehicle's computing resources, and the local computation time $\tau^l_{v,k}(t)$ is given as follows,

$$\tau^l_{v,k}(t) = \frac{b_{v,k}(1-\lambda_{v,k}(t))D_{v,k}(t)}{f^l_{v,k}(t)}. \tag{2}$$

To begin executing tasks in the VEC, the vehicle k must first upload the task to the target VEC server v. The returned data, as per many research studies, is frequently significantly smaller than the uploaded data. Consequently, we ignore the transmission time of the returned data. As a result, task latency includes data upload time as well as execution time in the VEC server. The data upload time is given by,

$$\tau^o_{v,k}(t) = \frac{\lambda_{v,k}(t)D_{v,k}(t)}{R_{v,k}(t)}. \tag{3}$$

Furthermore, the computation time in the VEC server is defined as follows:

$$\tau^c_{v,k}(t) = \frac{b_{v,k}\lambda_{v,k}(t)D_{v,k}(t)}{f_v(t)}. \tag{4}$$

As a result, the VEC's communication and processing delays are given by,

$$\tau^{tot}_{v,k}(t) = \tau^o_{v,k}(t) + \tau^c_{v,k}(t). \tag{5}$$

D. Queuing Model

Each vehicle maintains two queues locally; the queue for local computation is $Q^l_{v,k}(t)$ and the queue for offloading tasks to VEC v for processing is $Q^o_{v,k}(t)$. Each VEC v also maintains a queue $Q^c_{v,k}(t)$ for each connected vehicle.

The size of computation tasks calculated locally at the vehicle at each time slot is represented by $D^l_{v,k}(t)$ bits, and it is expressed as,

$$D^l_{v,k}(t) = \frac{\tau^l_{v,k}(t)f^l_{v,k}(t)}{b_{v,k}}. \tag{6}$$

Note, that the $f^l_{v,k}(t)$ can be adjusted according to the needs of the task but is limited by the maximum computing capacity of the vehicle $f^l_{\max}$. Therefore, the local computation queue is expressed by,

$$\begin{aligned} Q^l_{v,k}(t+1) = &\ \max\{Q^l_{v,k}(t) - D^l_{v,k}(t), 0\} \\ &+ (1-\lambda_{v,k}(t))D_{v,k}(t). \end{aligned} \tag{7}$$

Next, regarding the backlog of the offloading queue $Q^o_{v,k}$, let $D^o_{v,k}(t)$ denote the offloading task size. Then the amount of computation tasks that can be transmitted to VEC should satisfy the following constraint:

$$D^o_{v,k}(t) \le R_{v,k}(t)\tau^o_{v,k}(t),\ \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V}. \tag{8}$$

In addition, the local offloading queue $Q^o_{v,k}$ is,

$$Q^o_{v,k}(t+1) = \max\{Q^o_{v,k}(t) - D^o_{v,k}(t), 0\} + \lambda_{v,k}(t)D_{v,k}(t). \tag{9}$$

Now consider the edge server v, it maintains a queue for each vehicle k connected to it and allocates computational resources for these vehicles. Let, $f_v(t)$ be the computational resources assigned by the VEC v. We assume that the computational resource is allocated in a time-sharing manner. Therefore, the calculation size $D^c_{v,k}(t)$ is formulated as,

$$D^c_{v,k}(t) = \frac{\tau^c_{v,k}(t)f_v(t)\varphi_{v,k}(t)}{b_{v,k}}, \tag{10}$$

where, $\varphi_{v,k}(t)$ represents the computation time ratio that VEC v allocates to the vehicle k. Since it is a ratio,

$$\sum_{k=1}^{\mathcal{K}^v} \varphi_{v,k}(t) \le 1,\ \forall k \in \mathcal{K}^v,\ \forall v \in \mathcal{V}. \tag{11}$$

Next, let us define $Q^c_{v,k}$ as the computation backlog for vehicle k in VEC v. Therefore, the update of $Q^c_{v,k}$ is given by,

$$Q^c_{v,k}(t+1) = \max\{Q^c_{v,k}(t) - D^c_{v,k}(t), 0\} + D^o_{v,k}(t). \tag{12}$$

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

E. Energy Consumption Model

The system energy consumption consists of three components: energy spent on (1) local computation, (2) transmission, and (3) VEC computation. The energy consumption due to local computation is given by,

$$E^l_{v,k}(t) = \xi_k f^l_{v,k}(t)^3 \tau^l_{v,k}(t), \tag{13}$$

where, ξ_k is the computation energy consumption coefficient. Next, the transmission energy consumption $E^o_{v,k}(t)$ is,

$$E^o_{v,k}(t) = p_{v,k}(t) \cdot \tau^o_{v,k}(t). \tag{14}$$

Similarly, the energy consumption at the VEC v to compute the task of vehicle k is,

$$E^c_{v,k}(t) = \zeta_v f_v(t)^3 \varphi_{v,k}(t) \tau^c_{v,k}(t), \tag{15}$$

where, ζ_v represents the computation energy consumption coefficient. Hence, the system energy cost in time slot t is the sum of all these three,

$$E(t) = \sum_{v=1}^{V} \sum_{k=1}^{\mathcal{K}^v} \left[E^l_{v,k}(t) + E^o_{v,k}(t) + E^c_{v,k}(t) \right]. \tag{16}$$

F. Problem Formulation

This paper explores the best combination of task offloading and resource allocation policies to satisfy system energy minimization under queue stability and resource constraints. In order to achieve this, we develop a framework where an intelligent decision is made to either process the computing tasks locally at the vehicle or transmit them to the VEC for processing. The vehicular CPU-cycle frequency is a factor in adding tasks to the local queue. On the other end, the edge server allocates a fraction of CPU processing time for each vehicle associated with it, which determines the queue length at the VEC. Accordingly, the parameter vector for optimization is represented by,

$$\Omega(t) = \{\lambda(t), f^l(t), p^l(t), D^o(t), \varphi(t)\}.$$

The optimization problem is defined as $\mathcal{P}_E$,

$$\mathcal{P}_E : \min_{\Omega(t)} E = \lim_{T\to\infty} \frac{\sum_{t=0}^{T-1} \mathbb{E}\{E(t)\}}{T}, \tag{17}$$

$$\begin{aligned}
&\text{such that,}\\
&C1 : \lambda_{v,k}(t) \in \{0,1\}, \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C2 : 0 \le p_{v,k}(t) \le P^l_{\max}(t), \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C3 : 0 \le f^l_{v,k}(t) \le f^l_{\max}(t), \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C4 : D^o_{v,k}(t) \le R_{v,k}(t)\tau^o_{v,k}(t),\ \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C5 : D^o_{v,k}(t) \le Q^o_{v,k}(t),\ \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C6 : \sum_{k=1}^{\mathcal{K}^v} \varphi_{v,k}(t) \le 1,\ \forall k \in \mathcal{K}^v,\ \forall v \in \mathcal{V},\\
&C7 : \lim_{t\to\infty} \frac{\mathbb{E}[Q^l_{v,k}(t)]}{t} = 0,\ \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C8 : \lim_{t\to\infty} \frac{\mathbb{E}[Q^o_{v,k}(t)]}{t} = 0, \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C9 : \lim_{t\to\infty} \frac{\mathbb{E}[Q^c_{v,k}(t)]}{t} = 0, \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C10 : \tau^l_{v,k}(t) \le \tau_{v,k}(t), \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V},\\
&C11 : \tau^{tot}_{v,k}(t) \le \tau_{v,k}(t), \forall k \in \mathcal{K}^v, \forall v \in \mathcal{V}.
\end{aligned}$$

Here, the constraint $C1$ tells that either a task is offloaded or locally executed; constraint $C2$ indicates the wireless transmission power has to be less than the predefined maximum value while offloading the task; $C3$ implies the CPU-cycle frequency constraint; $C4$ shows that offloading rate has a limit; $C5$ tells that the queue length cannot be negative; $C6$ is the constraint regarding the limited resources at the VEC server; constraints $C7$–$C9$ guarantee the queue stability. The constraint conditions $C7$–$C9$ mean that the three queues, namely $Q^l_{v,k}(t+1)$, $Q^o_{v,k}(t+1)$, and $Q^c_{v,k}(t+1)$ are in mean rate stable. Its physical meaning is that the queue will not grow to infinity even after a very long period. Constraints $C10$ and $C11$ ensure that each arrival task is completed on time.

III. Lyapunov Optimization

A. Basics of Lyapunov Optimization

Lyapunov optimization uses a Lyapunov function to control a dynamic system optimally [32]. This is a powerful method for optimizing the stochastic process while ensuring its stability. The Lyapunov optimization is widely used to perform stochastic network optimization in queuing systems [33].

The key to utilizing Lyapunov optimization in offloading modelling is defining the drift and penalty [44]. In a scenario involving offloading, the drift could be the energy queue drift or the task queue drift, while the penalty is typically the offloading objective, such as the reduction of energy consumption or execution latency. In our offloading scenario, the drift indicates the queue backlog, and the penalty is the offloading objective. Then, by minimizing the drift-plus-penalty expression for each time slot, the optimal offloading decision and other parameters could be determined. In our work, Let $\boldsymbol{Q}(t) = \{Q^l_{v,k}(t), Q^o_{v,k}(t), Q^c_{v,k}(t)\}$ denote the current total queue vector, that consists of local execution queue, offloading queue, and the queue at the VEC server. The quadratic form of the Lyapunov function is considered here because it can handle dynamical systems with less complexity and can be computed easily and quickly [40]. As a result, we can derive the corresponding quadratic Lyapunov function $L(\boldsymbol{Q}(t))$ as follows:

$$L(\boldsymbol{Q}(t)) = \frac{1}{2} \sum_{v=1}^{V} \sum_{k=1}^{\mathcal{K}^v} \left[Q^l_{v,k}(t)^2 + Q^o_{v,k}(t)^2 + Q^c_{v,k}(t)^2 \right]. \tag{18}$$

Next, the Lyapunov drift is expressed as,

$$\Delta(\boldsymbol{Q}(t)) = \mathbb{E}\left[L(\boldsymbol{Q}(t+1)) - L(\boldsymbol{Q}(t)) | \boldsymbol{Q}(t) \right]. \tag{19}$$

Since we aim to minimize energy consumption as well as to guarantee queue stability, we focus on the following drift-plus penalty function [42],

$$\Delta_\Gamma(\boldsymbol{Q}(t)) = \Delta(\boldsymbol{Q}(\boldsymbol{t})) + \Gamma \mathbb{E}[E(t)|\boldsymbol{Q}(t)], \tag{20}$$

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

where, Γ is a tunable weight representing the relative importance of "energy consumption" compared to "queue stability." A larger Γ indicates the higher priority of the energy consumption and vice versa. For simplicity, we denote $\Delta_\Gamma(\boldsymbol{Q}(t))$ as Ψ_Γ hereafter.

Our next task is to minimize Ψ_Γ at each time slot to achieve the optimal energy cost while maintaining the queue stability. Under this situation, the drift-plus penalty function has the following upper bound,

$$\begin{aligned}&\Delta(\boldsymbol{Q}(t)) + \Gamma\mathbb{E}[E(t)|\boldsymbol{Q}(t)] \leq Z + \Gamma\mathbb{E}[E(t)|\boldsymbol{Q}(t)]\\&+\mathbb{E}\left[\sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} Q^l_{v,k}(t)[(1-\lambda_{v,k}(t))D_{v,k}(t) - D^l_{v,K}(t)]|\boldsymbol{Q}(t)\right]\\&+\mathbb{E}\left[\sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} Q^o_{v,k}(t)[\lambda_{v,k}(t)D_{v,k}(t) - D^o_{v,K}(t)]|\boldsymbol{Q}(t)\right]\\&+\mathbb{E}\left[\sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} Q^c_{v,k}(t)[D^o_{v,k}(t) - D^c_{v,K}(t)]|\boldsymbol{Q}(t)\right],\end{aligned} \tag{21}$$

where, Z is a positive constant and the upper bound for the expression $\mathbb{E}[E(t)|\boldsymbol{Q}(t)]$.

IV. Lyapunov-Based MADDPG Resource Management

The optimization problem (17) is non-convex. Since an analytical solution is not tractable, we use a machine-learning approach. RL is a promising solution to this problem since we don't have labelled data [34]. We first define joint task offloading and resource allocation problems to deploy RL. Here our goal is to minimize the consumed energy by determining the best possible offloading policy. Second, we model each vehicle as an agent and take actions based on local observations and interactions with the VEC system. These observations include computation ratio $\varphi(t)$ and queue length at the edge server. Finally, we employ Lyapunov-MADDPG (L-MADDPG) algorithm to solve this task offloading and resource allocation problem, as shown in Fig. 2. The details of edge server computation ratio $\varphi(t)$ are given in the following section.

A. Edge Server Computation

The status of the edge server computation is a crucial observation for our L-MADDPG model and is a significant constraint to our optimization statement. We denote the computation resource allocation subproblem as $\mathcal{P}_C$. Note that $\mathcal{P}_C$ is decoupled for each edge server. Edge servers solve $\mathcal{P}_C$ as in Algorithm 1 and send the information to the connected vehicles. Thus, $\mathcal{P}_C$ is given as,

$$\begin{aligned}\mathcal{P}_C: \min_{\varphi(t)} &\sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} \Gamma\zeta_v f_v(t)^3 \varphi_{v,k}(t)\tau^c_{v,k}(t)\\&- Q^c_{v,k}(t)\frac{\tau^c_{v,k}(t)f_v(t)\varphi_{v,k}(t)}{b_{v,k}},\end{aligned} \tag{22}$$

such that,

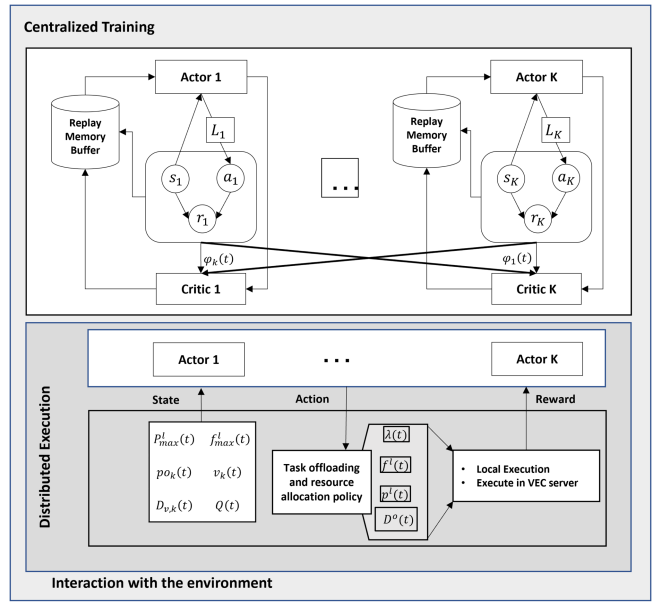


Fig. 2. Lyapunov-based MADDPG framework for VEC-assisted vehicular network.

$$\sum_{k=1}^{\mathcal{K}^v} \varphi_{v,k}(t) \leq 1,\ \forall k \in \mathcal{K}^v,\ \forall v \in \mathcal{V},$$

We introduce $Y_{v,k}(t)$ to solve $\mathcal{P}_C$ and define it as,

$$Y_{v,k}(t) = \Gamma\zeta_v f_v(t)^3 \tau^c_{v,k}(t) - Q^c_{v,k}(t)\frac{\tau^c_{v,k}(t)f_v(t)}{b_{v,k}}. \tag{23}$$

The subproblem $\mathcal{P}_C$ is a kitbag problem, where the kitbag capacity is one and $Y_{v,k}(t)$ can be seen as the unit value of the item. Note the ideal solution for the problem is to select the item with the least negative unit value until the kitbag is packed or there are no non-positive items. To optimize the $\varphi_{v,k}(t)$, we first initialize the available computation resources $\eta_v(t)$ of the VEC server v as 1. Second, calculate the values $Y_{v,k}(t)$ and sort them in increasing order. Third, according to the sorted order, we allocate the computational resources to a set of vehicles $\mathcal{K}^v$. Therefore, the computational resource allocation at the VEC server v is given as,

$$\varphi^*_{v,k}(t) = \begin{cases}\min\left\{\frac{Q^c_{v,k}(t)b_{v,k}}{\tau^c_{v,k}(t)f_v(t)}, \eta_v(t)\right\}, & \text{if } Y_{v,k}(t) \leq 0,\ k = k^*\\ 0, & \text{otherwise,}\end{cases} \tag{24}$$

where, $k^* = \arg\min Y_{v,k}(t)$, for $k \in \mathcal{K}^v$. Finally, we update $\eta_v(t)$ according to the relationship $\eta_v(t) = \eta_v(t) - \varphi^*_{v,k}(t)$.

B. Deep Reinforcement Learning-Based Solution Method

The deep reinforcement learning method is then used to solve the proposed problem. We first formulate the problem as a Markov decision process (MDP) to accurately describe the offloading and resource allocation decision processes. In our MDP

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

formulation, each vehicle is modeled as an agent, and actions are taken based on local observations and interactions with the VEC system. These observations include the computation ratio $\varphi(t)$ and queue length at the edge server, which are critical variables for determining resource allocation policies at the centralized network. To solve the MDP, we use the L-MADDPG algorithm, which is a deep reinforcement learning algorithm that can effectively handle non-convex and stochastic optimization problems. Our proposed L-MADDPG algorithm can devise efficient task scheduling and resource allocation strategies to dynamically adjust the decision variables to minimize system energy consumption without relying on prior system state information. We train the L-MADDPG model offline to ensure that each agent efficiently schedules and allocates resources. Here we present the elements of the MDP, including the state space, action space, and reward function.

1) *State Space:* Let the state space of the vehicle k at time t be $s_k(t)$. The state space includes information such as speed $\nu_k(t)$. Let $po_k(t)$ denote the longitude and latitude coordinates of the vehicle at time t. Note, the arrived task at the vehicle k is $D_{v,k}(t)$ and the maximum resources at vehicle k is $\{P^l_{\max}(t), f^l_{\max}(t)\}$ and $\boldsymbol{Q}(t)$ is the current queue vector. Therefore, the state space at time t is given as,

$$\boldsymbol{S}(t) = (\boldsymbol{s}_1(t), \ldots, \boldsymbol{s}_k(t), \ldots, \boldsymbol{s}_K(t)), \tag{25}$$

where,

$$\boldsymbol{s}_k(t) = [po_k(t), \nu_k(t), D_{v,k}(t), P^l_{\max}(t), f^l_{\max}(t), \boldsymbol{Q}(t)].$$

We regard the vehicle's speed and position to be input states to the action space, so changes in speed and position are continuously updated.

2) *Action Space:* According to the current policy μ_k, and the corresponding state, each VEC server chooses an action from its action space. Actions include task offloading policy $\lambda_{v,k}(t)$, transmission power control policy $p^l_{v,k}(t)$, local computing resource allocation policy $f^l_{v,k}(t)$ etc. Also, these actions consider the size of the offloaded tasks $D^o_{v,k}(t)$. Therefore, the action space at time t is given as,

$$\boldsymbol{A}(t) = (\boldsymbol{a}_1(t), \ldots, \boldsymbol{a}_k(t), \ldots, \boldsymbol{a}_K(t)), \tag{26}$$

where,

$$\boldsymbol{a}_k(t) = [\lambda_{v,k}(t), p^l_{v,k}(t), f^l_{v,k}(t), D^o_{v,k}(t)].$$

3) *Reward Function:* Our main objective is to minimize vehicles' energy consumption while meeting the task execution delay threshold. We achieve this goal by appropriately allocating the computational resources of the VEC servers while meeting the queue stability. Therefore, we set rewards to accelerate the training speed while meeting the constraints. After an agent performs an action $a_k(t)$, the environment provides feedback in the form of a reward. In this scenario, the reward corresponds to the quantity of energy consumed by the user terminal following the execution of the resource's allocation and offloading scheme. This paper aims to minimize the energy consumption of each vehicle, subject to the availability of system resources and the stability of the queue. Consequentially, the reward is determined in accordance with the constraints and objective, including the incentive for wireless resources, computation ratio, and energy consumption, which are pursued sequentially [34]. The reward function is defined as follows, assuming that the state of the vehicle k satisfies the constraints $(C1)$–$(C6)$,

$$\begin{aligned}
r_k(t) = e_1 &\\
&+ w_1\Bigg(\min_{\lambda(t)} \sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} Q^l_{v,k}(t)(1-\lambda_{v,k}(t))D_{v,k}(t)\\
&+ Q^o_{v,k}(t)\lambda_{v,k}(t)D_{v,k}(t)\\
&- Q^l_{v,k}(t)\frac{\tau^l_{v,k}(t)f^l_{v,k}(t)}{b_{v,k}}\Bigg)\cdot \Lambda_{(Q^o_{v,k}(t)\geq Q^l_{v,k}(t))},\\
&+ w_2\Bigg(\min_{f^l(t)} \sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} \Gamma\xi_{v,k} f^l_{v,k}(t)^3\tau^l_{v,k}(t)\\
&- Q^l_{v,k}(t)\frac{\tau^l_{v,k}(t)f^l_{v,k}(t)}{b_{v,k}}\Bigg)\cdot \Lambda_{(0\leq f^l_{v,k}(t)\leq f^l_{\max}(t))},\\
&+ w_3\Bigg(\min_{p^l(t),D^o(t)} \sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} \Gamma p_{v,k}(t)\frac{D^o_{v,k}(t)}{R_{k,v}(t)}\\
&- Q^o_{v,k}(t)D^o_{v,k}(t)\Bigg)\\
&\cdot \Lambda_{(D^o_{v,k}(t)\leq R_{v,k}(t)\tau^o_{v,k}(t) \text{ and } D^o_{v,k}(t)\leq Q^o_{v,k}(t))},\\
&+ w_4\Bigg(\min_{\varphi(t)} \sum_{v=1}^{V}\sum_{k=1}^{\mathcal{K}^v} \Gamma\zeta_v f_v(t)^3\varphi_{v,k}(t)\tau^c_{v,k}(t)\\
&- Q^c_{v,k}(t)\frac{\tau^c_{v,k}(t)f_v(t)\varphi_{v,k}(t)}{b_{v,k}}\Bigg)\\
&\cdot \Lambda_{(\sum_{k=1}^{\mathcal{K}^v}\varphi_{v,k(t)}\leq 1)},
\end{aligned} \tag{27}$$

where, e_1, w_1, w_2, w_3, and w_4 are experimental parameters and $\Lambda_{(\cdot)}$ indicates that if the condition $(\cdot)$ is not satisfied, the value is -1; otherwise, the value is 0.

Therefore, after taking action, $a_k(t)$, if the vehicle state $s_k(t)$ satisfies all constraints $(C1)-(C11)$, then the reward function $r_k(t)$ is,

$$r_k(t) = e_2 + w_5 \cdot \exp(-E(t)), \tag{28}$$

where, e_2 and w_5 are experimental parameters. On the other hand, if the action $a_k(t)$ *does not* satisfy at least any one of the constraints among $(C7)-(C11)$, then the reward $r_k(t)$ is,

$$r_k(t) = w_6 \cdot \exp(-E(t)) + p, \tag{29}$$

where, w_6 is the experimental parameter, and p is the penalty since the queue stability is not maintained. Finally,

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

the total reward r_t, over all the vehicles is given as,

$$r_t = \sum_{k=1}^{K} r_k(t). \tag{30}$$

C. Lyapunov-Based MADDPG Framework

The proposed Lyapunov-based MADDPG framework is presented in Fig. 2, where each agent owns two networks: an actor-network π and a critic network $\mathbb{Q}$. Both the actor and critic networks are fully connected [46], [47]. The vehicle state information $po_k(t)$, $\nu_k(t)$, $D_{v,k}(t)$, $P^l_{\max}(t)$, $f^l_{\max}(t)$, and $\boldsymbol{Q}(t)$ are the inputs to the actor-network. The actor-network calculates the action to be executed based on the agent's state. In contrast, the critic network estimates the expected return from the current state to obtain an optimal policy. The VEC server queue and its available computation resources are observed in our work for further learning.

Algorithm 1 estimates the computation time ratio $\varphi_{v,k}(t)$ at the VEC server, which is used to train a centralized critic network. The critic network randomly reads the data from the experience replay buffer. The randomness avoids correlations among the training data and makes the training process unbiased [35]. Note that we use centralized training and distributed execution. During the centralized training phase, the actor-network only observes its own data, whereas the critic-network acquires the actions and observations of all agents. The critic network is not involved in the distributed execution phase, and each agent requires only the actor-network. By interacting with the environment, each agent can devise task offloading and resource allocation policies, thereby optimizing the parameters $\lambda_{v,k}(t), p^l_{v,k}(t), f^l_{v,k}(t), \varphi_{v,k}(t)$ and $D^o_{v,k}(t)$.

The centralized training process is composed of K agents, whose network parameters are $\boldsymbol{\theta} = \{\theta_1, \ldots, \theta_K\}$. We define $\boldsymbol{\mu} = \{\mu_{\theta 1}, \ldots, \mu_{\theta k}\}$ where μ_j is the set of deterministic policies for all agents. Therefore, for the deterministic policy μ_k of agent k, the gradient with Lyapunov constraint can be written as:

$$\nabla_{\theta_k} J(\mu_k) \simeq \mathbb{E}_{(\boldsymbol{S},a)\sim\boldsymbol{\mathcal{D}}}\Bigg[\nabla_{\theta_k \mu_k}(a_k|s_k)\nabla_{a_k}\mathbb{Q}^{\boldsymbol{\mu}}_k (\boldsymbol{S}, a_1, \ldots, a_K)|a_k = \mu_k(s_k) + \beta\Delta\Psi_\Gamma(\mu_k)\Bigg], \tag{31}$$

where, $\boldsymbol{\mathcal{D}}$ is the experience replay buffer, which contains the series of $(\boldsymbol{S}, \boldsymbol{A}, \boldsymbol{S}', R)$. Here $\beta \in [0, 1]$ and $\Delta\Psi_\Gamma(\mu_k)$ are the drift-plus penalty functions associated with all the old and new policies of $\boldsymbol{\mu}_k$. At training, the drift-plus penalty function will be parameterized by Γ, which is trained to minimize the loss function of the critic network and updated as follows:

$$\mathcal{L}_{oss}(\theta_k) \simeq \mathbb{E}_{(\boldsymbol{S},\boldsymbol{A},R,\boldsymbol{S}')}\left[\frac{1}{2}\Psi_\Gamma((\boldsymbol{S}, a_1 \cdots, a_K) - \Psi_{\Gamma(target)})^2\right], \tag{32}$$

where,

$$\Psi_{\Gamma(target)} = r_t + \gamma\Psi_\Gamma(\boldsymbol{S}', a'_1, \ldots, a'_K)|a'_{i=\mu'_i(s_i)}, \tag{33}$$

Algorithm 1: $\varphi_{v,k}(t)$ Optimization at the VEC Server.

Require: The amount of arrival computation tasks $D_{v,k}(t)$, and queue backlogs $\boldsymbol{Q}(t)$.
Output: The optimal decision of $\varphi_{v,k}(t)$.

```
for v ∈ V do
    Initialize the available computation resources
    η_v(t)=1;
    Calculate Y_{v,k}(t) and sort them in increasing order.
    for k ∈ K^v do
        Allocate the computation resources φ*_{v,k}(t) to set
        of
        vehicles'according to (24) in sorted order.
        Update η_v(t) according to
        η_v(t) = η_v(t) − φ*_{v,k}(t).
    end for
end for
```

and γ is the discount factor. The action network is updated by minimizing the policy gradient of the agent, which can be expressed as,

$$\nabla_{\theta_k} J \approx \frac{1}{\mathbb{S}}\sum_i \Bigg[\nabla_{\theta_k \mu_k}(s^i_k)\nabla_{a_k}\Psi_\Gamma (\boldsymbol{S}^i, a^i_1, \ldots, a^i_K)|a_k = \mu_k(s^i_k)\Bigg], \tag{34}$$

where, $\mathbb{S}$ is the mini-batch size, and i is the index of the samples. Algorithm 2 details the L-MADDPG training algorithm, while Algorithm 3 describes the execution algorithm.

V. Performance Evaluation

A. Parameter Setting

All experiments were carried out in Colab with GPU Tesla k80, CPU @ 2.20 GHz, 13 GB of RAM, and a hard disk of 108 GB. All the models are built using Keras API. The proposed L-MADDPG, MADDPG, DDPG, PPO and TRPO architectures are implemented on the TensorFlow platform (v1.14.0) [36]. We have chosen DDPG, in contrast to PPO and TRPO, which do not support off-policy correction. DDPG can also generate broad context profit from statistical data [43]. Hyperparameters for L-MADDPG are as follows, the learning rate is 0.0001, and the batch size is 128. The activation function is ReLu, and Adam optimizer is used. The hyperparameters of the proposed L-MADDPG are shown in Table II.

B. Simulation Results

This section presents simulation results to validate the performance of the proposed scheme. Specifically, we use PTV Vissim [37], a traffic simulation software, to simulate vehicle mobility on a two-lane and two-way highways road. The behaviour type is "Freeway". Environment states are obtained according to the vehicles' position information. We present the simulation results of the optimization algorithm with parameters such as

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Training of Proposed L-MADDPG.

Initialize Parameters: θ_k, μ_k
Initialize each agent's actor and critic evaluations and target networks to explore actions for the training stage.
Consider $\varphi_{v,k}(t)$ for the training of the centralized critic network.
Initialize the size of each agent's replay memory buffer.
Initialize the learning rate with random parameter ϖ.
for $t = 1 : T$ **do**
Receive initial state S.
  for $v \in V$ do
    for $k \in \mathcal{K}^v$ do
      Execute actions $a_k(t)$ and obtain new state $s'_k(t)$
      if the $s'_k(t)$ does satisfy constraints $C1 - C6$ in (17);**then**
        Obtain reward of vehicle k based on (27)
      else if the $s'_k(t)$ does satisfy all the constraints $C1 - C11$ in (17);**then**
        Obtain reward of vehicle k based on (28)
      else if the $s'_k(t)$ does not satisfy at least one of the constraints $C7 - C11$ in (17);**then**
        Obtain reward of vehicle k based on (29).
    end for
    Obtain the action A, new state S' and reward R;
    Store (S, A, S', R) in replay buffer $\boldsymbol{\mathcal{D}}$.
  end for
  for $k \in \mathcal{K}^v$ do
    Sample a random mini-batch of $\mathbb{S}$ samples.
    $(\boldsymbol{S}^i, \boldsymbol{A}^i, R^i, \boldsymbol{S}'^i)$ in replay buffer $\boldsymbol{\mathcal{D}}$ assign rewards.
    Calculate drift-plus penalty function $\Delta\Psi_\Gamma(\mu_k)$. Record current reward $r_k(t)$.
    Compute overall reward r_t based on (30). Update Lyapunov-based MADDPG networks.
    Update the critic network by minimizing the loss function using (32).
    Update the actor network using the sampled policy gradient (34).
  end for
Update the target network parameters of each vehicle using,
$\theta'_k \leftarrow \varpi\theta_k + (1-\varpi)\theta'_k$,
$\mu'_k \leftarrow \varpi\mu_k + (1-\varpi)\mu'_k$.
end for

Algorithm 3: Execution of Proposed L-MADDPG.

Require: For all vehicles Initialize the positions, speed, task queue, computation resources and wireless resources. Input the $\varphi_{v,k}(t)$ of the VEC server. Input the weights of actor network and execution time T.
Initialize the state space S, the action space A and the reward R of all vehicles.
Output: $A(t)$
for $t = 1 : T$ **do**
  for each vehicle $\mathcal{K}^v = \{1, \ldots, k, \cdots K^v\}$ **do**
    Obtain the current state information of vehicle k, $s_k(t)$ and execute action $a_k(t)$ based on the weights of actor network of vehicle k;
    Obtain the new state $s'_k(t)$ and reward $r_k(t)$;
    Store action $a_k(t)$, state $s_k(t)$ and $r_k(t)$ into A_t, S_t, and R_t respectively;
    Return $A(t)$;
  end for
end for

TABLE II
HYPERPARAMETERS FOR THE PROPOSED L-MADDPG ALGORITHM

Hyperparameter	Value
Number of vehicles	1-24
Length of considered road segment	2200m
$f^l_{v,k}(t)$	0.5 $\sim$ 5.5 GHz
$f_v(t)$	2 $\sim$ 21.8 GHz
$D_{v,k}(t)$	[0, 1.8] Mb
δ	1 ms
Length of each episode	60000 ms
Number of episodes	5000
Length of each step	1000ms
Mini batch Size	128
Layer type of Actor and Critic	Fully connected
Learning rate of Actor and Critic	0.0001
ϵ-greedy value	0.99
Experience Replay Memory Size	50,000
Packet size	512 bytes
Speed	30-120 kms/hr
γ	0.95
Parameters of reward	e_1=-0.4, e_2=-0.3, w_1, w_2, w_3, w_4, w_5=0.5, w_6= 0.6
Penalty (p)	-0.6

Γ, arrival rate, and $\varphi(t)$. Also, we compare the performance of the proposed L-MADDPG algorithm with existing PPO, TRPO, DDPG and pure MADDPG algorithms in terms of convergence performance of the average reward, queue length and energy consumption. The length of the queue is measured in bits. In our simulation, the queue length threshold has been maintained at 5 MB. In our simulation part, the state information is updated for each step.

1) Impact of Parameter Γ*:* Fig. 3 depicts the impact of the trade-off parameter Γ on the queue length for various traffic loads for the proposed algorithm. The average queue length increases with Γ. When Γ increases, the execution of tasks may be delayed to decrease energy consumption. This shows that the smaller the Γ, the higher the priority of the queue stability in the proposed algorithm. Fig. 4 illustrates the impact of the trade-off parameter Γ on average energy consumption. It shows that the average energy consumption for the proposed L-MADDPG decreases with increasing Γ. Combining these two figures, we can conclude that with the correct choice of Γ, our proposed

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

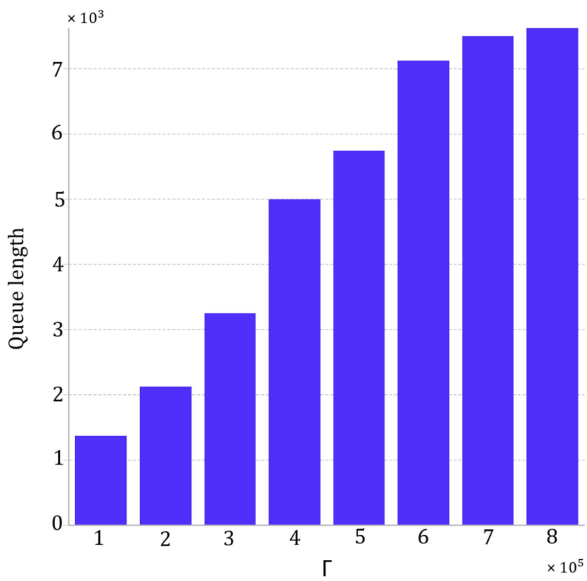


Fig. 3. Queue length versus the control parameter Γ.

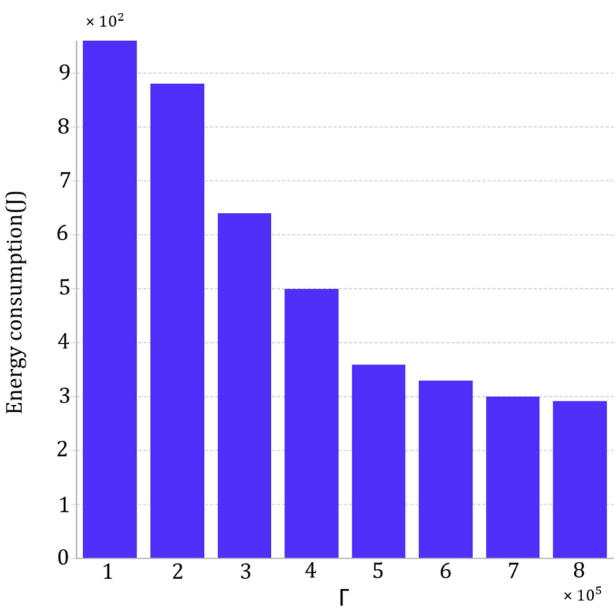


Fig. 4. Energy consumption versus the control parameter Γ.

L-MADDPG algorithm can optimize the system's energy costs while maintaining queue stability.

2) Impact of Task Arrival Rate: The queue length and system energy consumption for different task arrival rates is shown in Figs. 5 and 6, respectively. Since the task arrival rate is α times $D_{v,k}(t)$, the arrival rate of the data was changed by varying α. We considered values of 0.8, 1.0, and 1.2 for α. Fig. 5 shows that although the queue length increase with the task arrival rate, it does stabilize. This is a proof that the proposed algorithm guarantees queue stability.

Fig. 6 shows the accumulated energy consumption for various traffic loads. We can see that the average energy consumption increases with a rise in the arrival rate which is natural.

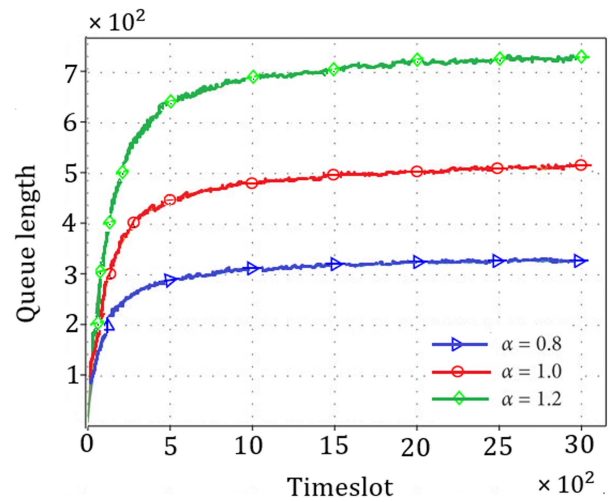


Fig. 5. Queue length versus timeslot with different arrival rates.

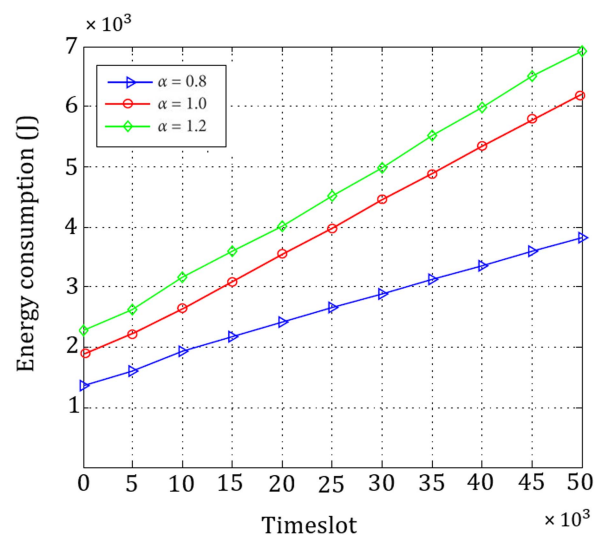


Fig. 6. Accumulated energy consumption versus timeslot with different arrival rates.

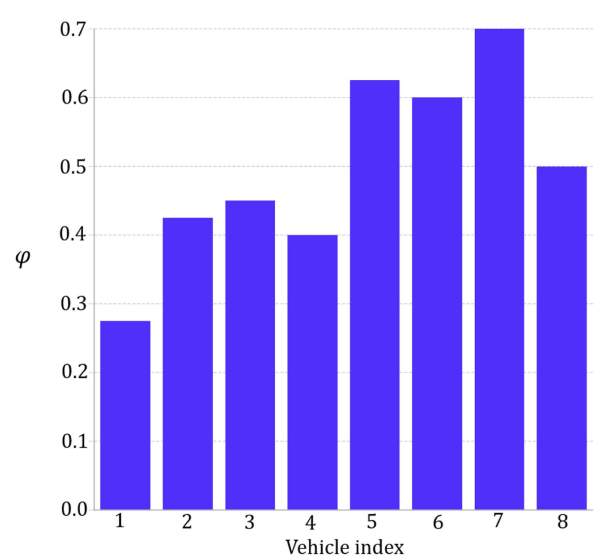


Fig. 7. Ratio of the computation resource allocated to vehicles.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

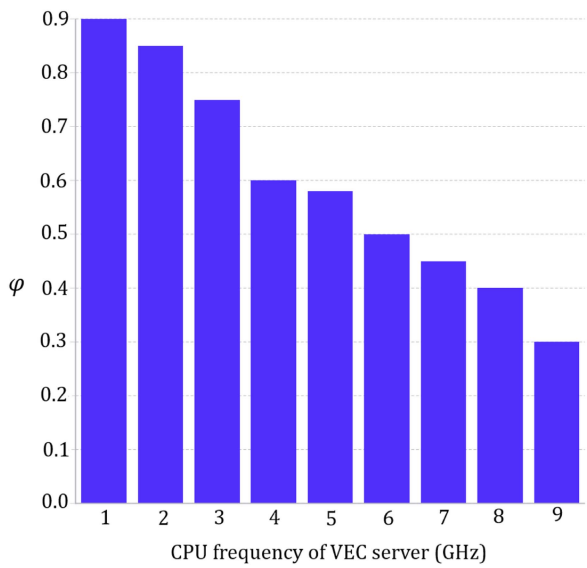


Fig. 8. Ratio of the computation resource allocated to a single vehicle with the increasing computation resource of the VEC server.

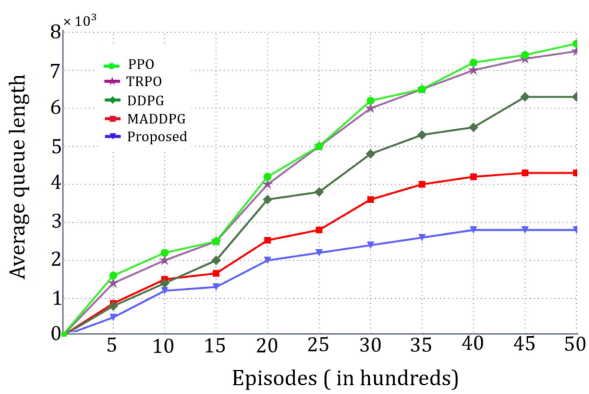


Fig. 9. Average queue length with different algorithms.

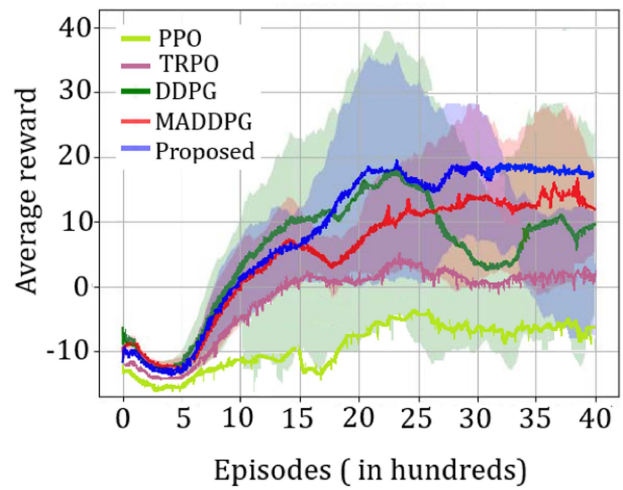


Fig. 10. Learning curves of different algorithms.

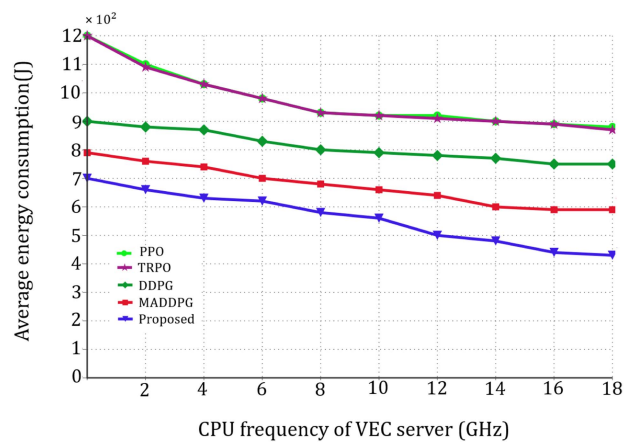


Fig. 11. Energy consumption of each vehicle Vs VEC computing resources.

3) Impact of $\varphi(t)$: Fig. 7 presents the computation resources allocated by the VEC server to vehicles 1 to 8 in the proposed algorithm. Here, Γ is 7×10^5. Note that vehicles five and seven have more data to offload than other vehicles, so we find a significant increase in the computation ratio for these two vehicles compared with others.

Fig. 8 depicts the computation allocation ratio with the increased VEC server resources. Here, we consider a single vehicle. We can see a decrease in the allocation ratio as the CPU speed increases. This occurs because growth in the VEC processing capability enables it to compute tasks quickly and accommodate more vehicles, reducing the allocation ratio.

4) Comparison Experiments: Fig. 9 shows the convergence behaviour of the average queue length with different algorithms when the number of vehicles is ten. The proposed algorithm reaches a lower and more stable queue length value validating the effectiveness of the Lyapunov training process. This figure proves that the proposed L-MADDPG algorithm meets the optimal policy earlier than pure MADDPG and other algorithms.

In Fig. 10, we present the learning curve performance of all three algorithms. As the training episodes increase, the average reward rises gradually, preserving a stable positive reward for both the proposed and pure MADDPG algorithms. The figure shows that the DDPG can produce very good results in the early stages of episodes, but it can later become unstable and heavily reliant on searching for the correct hyperparameters for the current task. However, the average reward for DDPG fluctuates more due to its sensitivity to exploration and learning rates. Our proposed algorithm shows the least fluctuation due to its more stable training process.

Fig. 11 shows the effect of the size of VEC computing resources on energy consumption. It plots the average energy consumption when the computing resources of VEC servers are from 2 to 18 GHz. The average energy consumption decreases with the increase of VEC computing resources. This is because the increased VEC processing capability decreases the usage of local computing resources at each vehicle. Hence, the energy consumption of each vehicle reduces gradually.

Fig. 12 shows the effect of the number of vehicles on their energy consumption. This is done for the VEC server frequency of 12 GHz. We can observe that the average energy consumption increases with the number of vehicles. This is because the VEC computing resources allocated to each user decrease as the number of users increases, resulting in more local computing

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

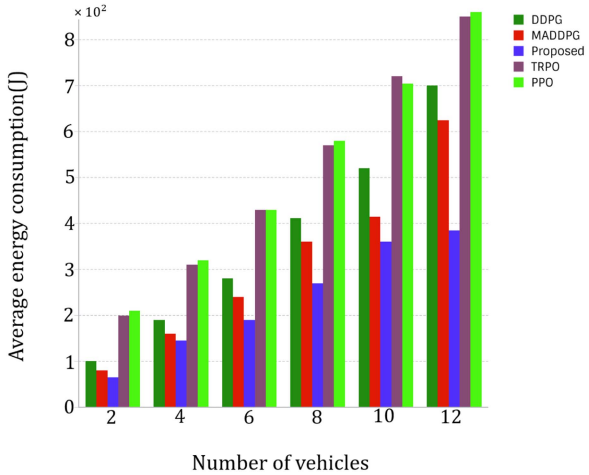


Fig. 12. Average energy consumption of each vehicle for different numbers of total vehicles.

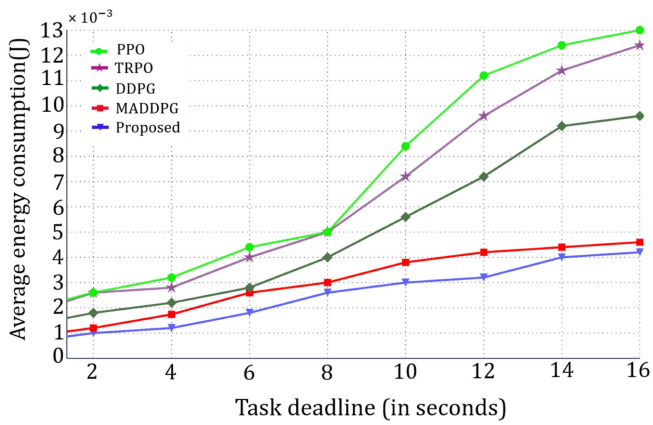


Fig. 13. Average energy consumption in different task deadline.

resources for each vehicle. Therefore, the energy consumption of the user terminals increases.

Fig. 13 depicts the connection between deadlines and average energy consumption. The average energy consumption of all algorithms (including PPO, TRPO, and DDPG) will increase proportionally as task deadlines are extended. As the deadline for a task approaches, the majority of tasks are completed, leading to an increase in the average amount of energy consumed. However, both our proposed method and pure MADDPG increase energy consumption prior to stabilization. Since the latency requirement is less stringent, users can request more assistance from the VEC server by uploading more data and save energy on data processing. In addition, as the required latency approaches a certain threshold, the energy consumption reduction relative to latency will diminish gradually. Thus, increasing latency has a negative impact on energy consumption. Consequently, Fig. 13 demonstrates the trade-off between energy consumption and latency.

C. Complexity Analysis

This section discusses the complexity of our proposed algorithm to solve $\mathcal{P}_E$. We treat $\mathcal{P}_C$ as a subproblem of $\mathcal{P}_E$ and solve it iteratively. First, the offloading ratio for $\mathcal{P}_E$ is computed, and the complexity of solving $\varphi_{v,k}(t)$ is $\mathcal{O}(K)$, which is linear with the number of users K. Second, to solve $\mathcal{P}_E$, we employ a three-layer iteration. The first layer (innermost) iteration consists of K users and receives input from $A_k(t) = \mathcal{O}(K)$, where the computational complexity is $\mathcal{O}(K^2)$. We have variables derived from the number of users and VEC servers at the second layer, so the computing complexity is given by $\mathcal{O}(KV)$. Consequently, the total computational complexity is $(\mathcal{O}(K^3) + \mathcal{O}(K)) \times \mathcal{O}(KV)$, which can be abbreviated as $(\mathcal{O}(K^4 V) + \mathcal{O}(K^2 V))$.

VI. Conclusion

In this paper, we proposed a vehicular edge computing framework that offloads computation-intensive tasks from vehicles to edge servers to save energy and enable fast execution. However, this offloading must be done intelligently to prevent exponential growth of queues at the edge server. Our optimization problem is non-convex and has constraints in transmission power, data rate, radio resources, and edge server capacity. To address this challenge, we proposed the Lyapunov-based multi-agent DDPG algorithm (L-MADDPG) that minimizes energy consumption while maintaining queue stability. L-MADDPG performs online offloading and resource allocation in vehicular networks, reducing system energy costs while ensuring stable performance. Our computer experiments demonstrate that L-MADDPG is very stable and effectively manages the queue length at the edge server, consuming less energy than existing algorithms. In future work, we plan to investigate task offloading and resource management in vehicular networks using a federated learning environment.

References

[1] F. Tang, Y. Kawamoto, N. Kato, and J. Liu, "Future intelligent and secure vehicular network towards 6G: Machine-learning approaches," *Proc. IEEE*, vol. 108, no. 2, pp. 292–307, Feb. 2020.

[2] A. S. Kumar, L. Zhao, and X. Fernando, "Multi-agent deep reinforcement learning-empowered channel allocation in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 2, pp. 1726–1736, Feb. 2022.

[3] K. Zhang, Y. Mao, S. Leng, Y. He, and Y. Zhang, "Mobile edge computing for vehicular networks: A promising network paradigm with predictive off-loading," *IEEE Veh. Technol. Mag.*, vol. 12, no. 2, pp. 36–44, Jun. 2017.

[4] L. Pu, X. Chen, G. Mao, Q. Xie, and J. Xu, "Chimera: An energy efficient and deadline-aware hybrid edge computing framework for vehicular crowdsensing applications," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 84–99, Feb. 2019.

[5] S. Raza, S. Wang, M. Ahmed, and M. R. Anwar, "A survey on vehicular edge computing: Architecture, applications, technical issues, and future directions," *Wireless Commun. Mobile Comput.*, vol. 2019, pp. 1–19, Jul. 2019.

[6] J. Gao, M. Li, L. Zhao, and X. Shen, "Contention intensity based distributed coordination for V2V safety message broadcast," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12288–12301, Dec. 2018.

[7] M. Li, J. Gao, L. Zhao, and X. Shen, "Deep reinforcement learning for collaborative edge computing in vehicular networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1122–1135, Dec. 2020.

[8] W. Zhan, C. Luo, J. Wang, G. Min, and H. Duan, "Deep reinforcement learning-based computation offloading in vehicular edge computing," in *Proc. IEEE Glob. Commun. Conf.*, Waikoloa, HI, USA, 2019, pp. 1–6.

[9] H. A. Shah and L. Zhao, "Multi-agent deep reinforcement learning based virtual resource allocation through network function virtualization in Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3410–3421, Mar. 2021.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

[10] Q. Zhang, H. Dong, and W. Pan, "Lyapunov-based reinforcement learning for decentralized multi-agent control," in *Proc. 2nd Int. Conf. Distrib. Artif. Intell.*, 2020, pp. 55–68.

[11] N. Moghadam, H. Li, H. Zeng, and L. Liu, "Lyapunov scheduling and optimization in network coded wireless multicast network," *IEEE Trans. Veh. Technol.*, vol. 67, no. 6, pp. 5135–5145, Jun. 2018.

[12] Z. Zhuang, J. Wang, Q. Qi, J. Liao, and Z. Han, "Adaptive and robust routing with Lyapunov-based deep RL in MEC networks enabled by blockchains," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2208–2225, Feb. 2021.

[13] S. Li, H. Chen, S. Lin, and N. Zhang, "Distributed task offloading and resource allocation in vehicular edge computing," in *Proc. Int. Conf. Space-Air-Ground Comput.*, 2021, pp. 13–18.

[14] Y. Sun et al., "Adaptive learning-based task offloading for vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3061–3074, Apr. 2019.

[15] S. Li, G. Zhu, and S. Lin, "Computation offloading with time-varying fading channel in vehicular edge computing," in *Proc. IEEE Globecom Workshops*, 2018, pp. 1–6.

[16] Y. Wang, X. Tao, X. Zhang, P. Zhang, and Y. T. Hou, "Cooperative task offloading in three-tier mobile computing networks: An ADMM framework," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2763–2776, Mar. 2019.

[17] S.-C. Lin, K.-C. Chen, and A. Karimoddini, "SDVEC: Software-defined vehicular edge computing with ultra-low latency," *IEEE Commun. Mag.*, vol. 59, no. 12, pp. 66–72, Dec. 2021.

[18] M. J. Neely, "Energy optimal control for time-varying wireless networks," *IEEE Trans. Inf. Theory*, vol. 52, no. 7, pp. 2915–2934, Jul. 2006.

[19] Y. Mao, J. Zhang, and K. B. Letaief, "A Lyapunov optimization approach for green cellular networks with hybrid energy supplies," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 12, pp. 2463–2477, Dec. 2015.

[20] Y. Zhang, C. Li, T. H. Luan, Y. Fu, W. Shi, and L. Zhu, "A mobility-aware vehicular caching scheme in content centric networks: Model and optimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3100–3112, Apr. 2019.

[21] A. Asheralieva and D. Niyato, "Game theory and Lyapunov optimization for cloud-based content delivery networks with device-to-device and UAV-Enabled caching," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10094–10110, Oct. 2019.

[22] Q. Gao, S. Lin, and G. Zhu, "Joint vehicular and static users multiplexing transmission with hierarchical modulation for throughput maximization in vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 9, pp. 3835–3847, Sep. 2020.

[23] S. C. Hung, X. Zhang, A. Festag, K.-C. Chen, and G. Fettweis, "Vehicle-centric network association in heterogeneous vehicle-to-vehicle networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 6, pp. 5981–5996, Jun. 2019.

[24] A. S. Kumar, L. Zhao, and X. Fernando, "Mobility aware channel allocation for 5G vehicular networks using multi-agent reinforcement learning," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.

[25] B. Lv, C. Yang, X. Chen, Z. Yao, and J. Yang, "Task offloading and serving handover of vehicular edge computing networks based on trajectory prediction," *IEEE Access*, vol. 9, pp. 130793–130804, 2021.

[26] X. Huang, L. He, X. Chen, L. Wang, and F. Li, "Revenue and energy efficiency-driven delay-constrained computing task offloading and resource allocation in a vehicular edge computing network: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8852–8868, Jun. 2022.

[27] X. Zhang, M. Peng, S. Yan, and Y. Sun, "Deep-reinforcement-Learning-Based mode selection and resource allocation for cellular V2X communications," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6380–6391, Jul. 2020.

[28] Y. Wei, F. R. Yu, M. Song, and Z. Han, "Joint optimization of caching, computing, and radio resources for fog-enabled IoT using natural actor–critic deep reinforcement learning," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2061–2073, Apr. 2019.

[29] M. Li, J. Gao, L. Zhao, and X. Shen, "Adaptive computing scheduling for edge-assisted autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5318–5331, Jun. 2021.

[30] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5404–5419, Aug. 2020.

[31] L. Huang, S. Bi, and Y. J. A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.

[32] M. J. Neely, "Stability and probability 1 convergence for queueing networks via Lyapunov optimization," *J. Appl. Math.*, vol. 2012, Jul. 2012, Art. no. 831909.

[33] L. Zheng and L. Cai, "A distributed demand response control strategy using Lyapunov optimization," *IEEE Trans. Smart Grid*, vol. 5, no. 4, pp. 2075–2083, Jul. 2014.

[34] X. Chen and G. Liu, "Energy-efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10843–10856, Jul. 2021.

[35] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multiagent actor–critic for mixed cooperative-competitive environments," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1–7.

[36] D. Paper, *State-of-the-art deep learning models in TensorFlow: Modern Machine Learning in the Google Colab Ecosystem*New York, NY, USA:Apress, Aug. 2021.

[37] "PTV Vissim." Accessed on: Mar. 15, 2023. 2022. [Online]. https://www.ptvgroup.com/en/solutions/products/ptv-vissim/

[38] W. Bao, H. Chen, Y. Li, and B. Vucetic, "Joint rate control and power allocation for non-orthogonal multiple access systems," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 12, pp. 2798–2811, Dec. 2017.

[39] K. Sadatdiynov, L. Cui, L. Zhang, J. Z. Huang, S. Salloum, and M. S. Mahmud, "A review of optimization methods for computation offloading in edge computing networks," *Digit. Commun. Netw.*, to be published, doi: 10.1016/j.dcan.2022.03.003.

[40] L. Zhang, Z. Zhao, Q. Wu, H. Zhao, H. Xu, and X. Wu, "Energy-aware dynamic resource allocation in UAV assisted mobile edge computing over social Internet of Vehicles," *IEEE Access*, vol. 6, pp. 56700–56715, 2018.

[41] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.

[42] H. Hu, W. Song, Q. Wang, R. Q. Hu, and H. Zhu, "Energy efficiency and delay tradeoff in an MEC-enabled mobile IoT network," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15942–15956, Sep. 2022.

[43] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3207–3214.

[44] W. Zhuang, Q. Ye, F. Lyu, N. Cheng, and J. Ren, "SDN/NFV empowered future IoV with enhanced communication, computing, and caching," *Proc. IEEE*, vol. 108, no. 2, pp. 274–291, Feb. 2020.

[45] Q. Ye, W. Shi, K. Qu, H. He, W. Zhuang, and X. Shen, "Joint RAN slicing and computation offloading for autonomous vehicular networks: A learning-assisted hierarchical approach," *IEEE Open J. Veh. Technol.*, vol. 2, pp. 272–288, 2021.

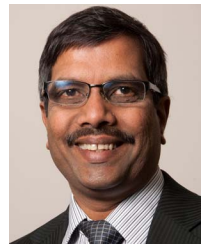
[46] Q. Zhang, H. Dong, and W. Pan, "Lyapunov-based reinforcement learning for decentralized multi-agent control," in *Proc. Int. Conf. Distrib. Artif. Intell.*, 2020, pp. 55–68.

[47] Z. Gao, L. Yang, and Y. Dai, "Fast adaptive task offloading and resource allocation via multi-agent reinforcement learning in heterogeneous vehicular fog computing," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6818–6835, Apr. 2023, doi: 10.1109/JIOT.2022.3228246.

Anitha Saravana Kumar (Student Member, IEEE) received the M.E. and Ph.D. degrees in electronics and communication engineering and information and communication from Anna University, Chennai, India, in 2004 and 2015, respectively. She is currently working toward the Ph.D. degree with the Department of Electrical, Computer, and Biomedical Engineering, Toronto Metropolitan University (formerly Ryerson University), Toronto, ON, Canada. She was a Professor with Visvesvaraya Technological University, Belgaum, India, till July 2019. Her research interests include the autonomous vehicular network, resource management, and deep learning. She has also authored more than 30 journal papers and conference papers. She was a reviewer for IEEE INTERNET OF THINGS JOURNAL and *Encyclopedia of Computer Science and Technology*, Inder science, Taylor and Francis publications. She was the TPC Member for the IEEE Globecom 2020, 2021, 2022 and ICC 2022 conferences. She received the Student Grant (WICE) from IEEE International Conference on Communications in 2021 and selected for Toronto Soapbox Science 2022.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.

Lian Zhao (Senior Member, IEEE) received the Ph.D. degree from the Department of Electrical and Computer Engineering (ELCE), University of Waterloo, Waterloo, ON, Canada, in 2002. In 2003, she joined the Department of Electrical and Computer Engineering, Toronto Metropolitan University (formerly Ryerson University), Toronto, ON, Canada. She has been an IEEE Communication Society (ComSoc) and IEEE Vehicular Technology (VTS) Distinguished Lecturer (DL). She was the recipient of the Best Land Transportation Paper Award from IEEE Vehicular Technology Society in 2016, Top 15 Editor Award in 2016 for IEEE Transaction on Vehicular Technology, Best Paper Award from the 2013 International Conference on Wireless Communications and Signal Processing (WCSP), and the Canada Foundation for Innovation (CFI) New Opportunity Research Award in 2005. She has been the Editor of IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, IEEE INTERNET OF THINGS JOURNAL, and IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY (2013–2021). She has been a Board of Governor (BoG) Committee Member since 2023. She was a Panel Expert in various federal, provincial, and international evaluation committees. She is a licensed Professional Engineer in the Province of Ontario.



Xavier Fernando (Senior Member, IEEE) was born in Sri Lanka. He received the B.Sc. Eng. (First Class Hons) degree in electrical and electronic engineering with the University of Peradeniya, Sri Lanka, in 1992, the master's degree in telecommunications from the Asian Institute of Technology, Thailand, in 1994, and the Ph.D. degree in electrical and computer engineering from the University of Calgary, Calgary, AB, Canada, in 2001, specializing in wireless communications in affiliation with TRLabs. From 1994 to 1997, he was an R&D Engineer for AT&T, Thailand. He is currently a Professor with Toronto Metropolitan (formerly Ryerson) University, Toronto, ON, Toronto. He was with the Ryerson University Board of Governors during 2011–2012. He was a Visiting Scholar with the Institute of Advanced Telecommunications (IAT), U.K., in 2008, and a MAPNET Visiting Fellowwith Aston University, Birmingham, U.K., in 2014. He has authored or coauthored one monograph (translated to Mandarin) and coauthored two more books. He has also coauthored more than 200 papers, many chapters and holds three patents. His research interests include wireless communication, vehicular networks and positioning. Dr. Fernando is a licensed Professional Engineer in Ontario. He was an IEEE Communications Society Distinguished Lecturer. He has been in the organizing/steering/technical program committees of many conferences. He was an Associate Editor for IEEE INTERNET OF THINGS JOURNAL and *IEEE Consumer Technology Magazine*. He was a Program Evaluator for ABET (USA). His work was the recipient of 30 awards and prizes so far including, Professional Engineers Ontario Award in 2016, IEEE Microwave Theory and Techniques Society Prize in 2010, Sarnoff Symposium Prize in 2009, Opto-Canada Best Poster Prize in 2003 and CCECE Best Paper prize in 2001. He was a finalist for Canadian Top Immigrant Award in 2012.

Authorized licensed use limited to: Kyunghee Univ. Downloaded on January 11,2026 at 08:24:26 UTC from IEEE Xplore. Restrictions apply.